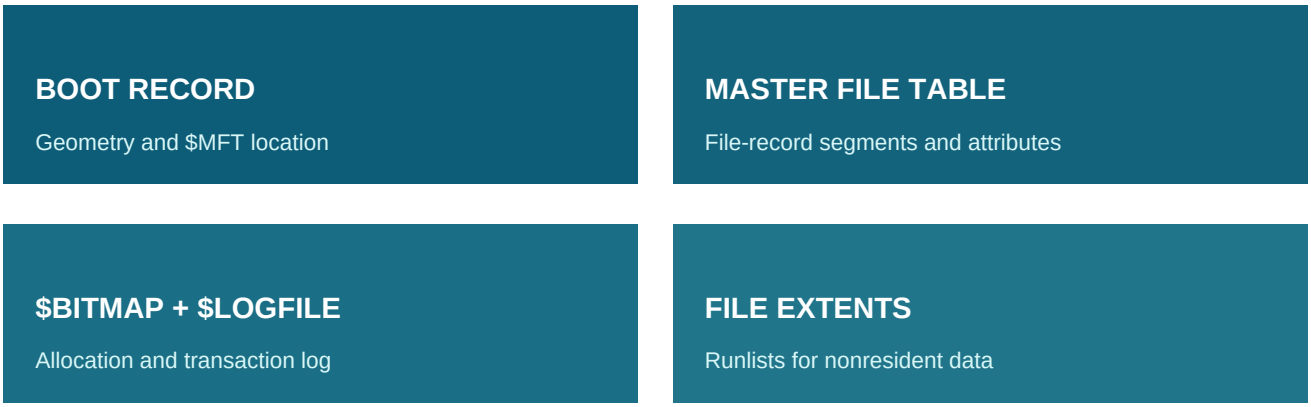
NTFS - Windows New Technology File System

OFFICIAL FORMAT / CHRONOLOGY / ENGINEERING

Microsoft’s Windows NT filesystem, built around an attribute-based Master File Table, metadata journals, allocation bitmaps and indexed directories.

This guide documents the original format’s technical architecture, history and independent source references. InfiltratorOS modules and install packages are implementation artefacts, not the authority for the historic on-disk definition.

CONCEPTUAL MEDIA MAP



CONTENTS

1. Historical chronology
2. Original on-disk architecture
3. Allocation and directories
4. Integrity and recovery
5. Design rationale and limitations
6. InfiltratorOS integration boundary

NTFS - Windows New Technology File System

HISTORICAL CHRONOLOGY

NTFS was introduced with Windows NT 3.1 in 1993, replacing the DOS-era FAT limitations for a security-oriented, multiuser 32-bit operating system. It was designed for large volumes, robust metadata management, access control, and extensible file representation. The New Technology lineage and its attribute-oriented record structures are central to understanding NTFS's design.

Later Windows releases evolved NTFS through additional attributes, indexes, reparse points, compression, encryption, change journals and transactional support. Third-party access tools and Linux's different NTFS drivers have varied in capability, but the on-disk format itself remains an independent Microsoft-defined system. A reader must distinguish volume revision and supported features, not assume identical semantics in every NTFS implementation.

NTFS - Windows New Technology File System

ORIGINAL ON-DISK ARCHITECTURE

The NTFS volume boot sector contains geometry and the starting logical cluster numbers of the Master File Table ($MFT) and its mirror. The $MFT contains variable-sized file record segments. Each file is represented by at least one record; even NTFS metadata files such as $MFT, $Bitmap and $LogFile are represented as files. Records contain typed attributes, which may be resident within the MFT record or nonresident elsewhere on the volume.

A file record starts with a signature and a fixup array used to detect incomplete sector writes. Attributes include $STANDARD_INFORMATION, $FILE_NAME, $DATA and other type-specific records. Nonresident attributes use mapping pairs or runlists to identify extents. Multiple named $DATA streams may belong to one file. NTFS’s internal structure must not be flattened to a single inode plus sequential sectors.

NTFS - Windows New Technology File System

ALLOCATION AND DIRECTORIES

$Bitmap tracks which volume clusters are allocated. The MFT's own allocation and growth are managed separately, with space reserved as an MFT zone to reduce fragmentation. Small files can fit entirely in resident $DATA attributes, while larger files have nonresident runs. Sparse and compressed attributes change logical-to-physical mapping and must be interpreted using attribute flags and allocated sizes.

Directories use indexed attributes such as $INDEX_ROOT and $INDEX_ALLOCATION, permitting tree-based lookup and sorted enumeration. Hard links are represented through multiple $FILE_NAME associations with file record identities. Security descriptors, object identifiers and reparse points add metadata semantics not captured by a simple FAT-like directory record.

NTFS - Windows New Technology File System

INTEGRITY AND RECOVERY

$LogFile contains the NTFS metadata transaction log, with redo and undo information used for consistency recovery. The volume dirty bit, update-sequence fixups, metadata redundancy and transactional logging serve different purposes. NTFS does not guarantee that all ordinary file data is journalled, and MFT mirroring covers only selected critical records.

Recovery requires considering log state, record sequence numbers, reference validity, MFT allocation, attribute lists, runlists and directory indexes. Chkdsk-like structural repair may be needed for damage beyond ordinary log replay. A cross-platform writer must not alter a hibernated or fast-startup Windows volume as though it were cleanly unmounted.

NTFS - Windows New Technology File System

DESIGN RATIONALE AND LIMITATIONS

NTFS invests more metadata and CPU work than FAT to support security, indexing, sparse data, resident attributes and independent metadata streams. The attribute system permits incremental evolution without changing every directory structure. Runlist compression and variable-sized resident records complicate a forensic implementation.

Features such as EFS encryption, compression, deduplication and reparse points can require Windows-specific services or metadata interpretation. A parser able to enumerate basic MFT entries is not automatically a safe complete writer, and a native Linux module must qualify each supported attribute family separately.

NTFS - Windows New Technology File System

INFILTRATOROS INTEGRATION BOUNDARY

NTFS.so is an independent format module and should not be confused with Microsoft's Windows kernel driver or with ntfs3 and NTFS-3G provider identities. Its usable operations and ABI must be proven against real MFT attributes, log state and independently formatted Windows volumes.

The historical design is documented from Microsoft developer materials. Our .deb, .run and optional kernel adapter install files into InfiltratorOS directories, but those paths have no bearing on actual NTFS disk geometry or transactional safety.

NTFS - Windows New Technology File System

PRIMARY SOURCE REFERENCES

These primary technical documents and near-contemporary references provide provenance for the design history and on-disk structures. For exact field sizes or a particular format revision, consult the original specifications and source code, not the conceptual cover illustration.

1. Microsoft: Master File Table, NTFS developer notes

https://learn.microsoft.com/en-us/windows/desktop/devnotes/master-file-table

2. Microsoft: Master File Table, local file systems overview

https://learn.microsoft.com/en-us/windows/win32/fileio/master-file-table

Our independent .so, optional .ko, build procedures and test evidence describe a separate implementation. The source-level history in this PDF cannot itself qualify production writes, boot, recovery or cross-platform compatibility.

NTFS / engineering detail

DISK READ PATH / ENGINEERING NOTES

1. Read the NTFS volume boot record and validate bytes per sector, sectors per cluster, volume bounds, and the logical cluster locations of the MFT and its mirror.

2. Locate file record zero in the Master File Table. Check its FILE signature and update sequence array before interpreting attributes; a torn multi-sector write must not be treated as valid metadata.

3. Walk typed attributes, respecting resident values inside the record and nonresident data runs elsewhere. Attribute-list entries can extend one file across additional MFT records.

4. Decode nonresident runlists as variable-length signed LCN deltas and unsigned run lengths. Virtual cluster numbers describe logical file order; a sparse run has no physical cluster allocation.

5. Resolve a directory through INDEX_ROOT and, where present, INDEX_ALLOCATION and its bitmap. Check ordering, references and child nodes instead of assuming a flat directory table.

6. Follow FILE_NAME links with full MFT record references and sequence numbers. Several names may legitimately refer to one file, and alternate data streams can share the same file identity.

7. Distinguish logical file size, initialized size and allocated size. Compression and sparse attributes make those values and the physical extent count differ.

8. Treat $MFT, $Bitmap, $LogFile, $BadClus and $Secure as distinct system files. Their recorded allocation and journal state form part of the volume interpretation.

NTFS / engineering detail

VALIDATION, RECOVERY AND COMPATIBILITY

1. Check the volume dirty state and any hibernation or Fast Startup evidence before considering writes from a non-Windows implementation. A clean directory listing is not sufficient.

2. Validate update-sequence fixups on every multi-sector MFT or index record. These are torn-write detectors, not replacement checksums for arbitrary file contents.

3. Check that every runlist terminates within volume bounds and that nonresident attributes have coherent VCN ranges. Reject negative or overflowed physical mappings.

4. Respect record sequence numbers when resolving a file reference. Reused MFT slots can make an old numeric record address point at a different file.

5. Replay of $LogFile is not the same operation as analysing $UsnJrnl. The former supports metadata consistency; the latter records change history for consumers.

6. Verify directory index entries against the file records they name. An apparently readable index page can still contain stale or inconsistent references.

7. Never assume $MFTMirr holds the entire MFT. It mirrors only a limited set of critical records, so recovery cannot be based on it as a complete backup.

8. For damaged media, preserve an image, report conflicting references and distinguish journal replay from destructive filesystem repair. The Microsoft format references remain authoritative.